

CPS3830 Machine Learning Project Proposal

项目题目

医疗问答检索场景下的 Reranker 对比研究：轻量级机器学习模型与开源重排序插件的比较

A Comparative Study of Lightweight ML Reranker and Open-Source Rerankers for Medical QA Retrieval

一、项目背景

在医疗问答系统中，检索阶段的质量会直接影响最终回答的可靠性。一个常见的 RAG 流程是：用户输入医疗问题，系统先从医疗问答数据库中检索出若干条候选资料，再将这些资料提供给大语言模型生成回答。

在这个过程中，如果检索到的资料不够相关，即使后续的大语言模型表达能力较强，也可能基于错误或不充分的 context 生成质量较低的回答。因此，对于医疗问答任务来说，如何从候选资料中选出更相关、更适合的问题上下文，是一个非常重要的问题。

目前基础检索方法通常依赖 embedding similarity，也就是将 question 和 context 转换成向量后计算相似度。但在医疗场景中，仅靠向量相似度可能不够精细。例如，不同疾病可能有相似症状，不同科室的问题也可能出现相似表达。因此，相似度高并不一定代表医学上真正相关。

本项目关注的不是完整 RAG 系统本身，而是 RAG 检索阶段中的一个独立机器学习问题：**如何对初步召回的医疗 context 进行重排序，使真正相关的资料排在更靠前的位置。**

二、项目定位

本项目将作为 CPS3830 Machine Learning 的独立课堂项目进行开发，并计划新建一个独立 GitHub repository，而不是直接在原有 RAG 项目中继续修改。

这样做的原因有三点：

第一，原有 RAG 项目是一个完整的个人项目，包含历史版本、医疗数据、RAG 系统实现和评测脚手架。为了避免与其他课程或老师合作项目混在一起，本项目需要保持独立边界。

第二，本项目的重点不是构建完整 RAG，而是研究一个机器学习子任务：question-context relevance prediction and reranking。这与 CPS3830 的课程目标更加一致。

第三，新建 repo 可以让项目结构更清晰，只保留本课程项目需要的数据处理、模型训练、开源 reranker 对比、实验结果和报告内容，便于老师查看和评分。

因此，本项目会复用原 RAG 项目中的医疗问答数据和部分实验思路，但不会把原 RAG 系统作为本项目成果提交。本项目的主体是一个独立的 reranker comparison pipeline。

三、项目动机

最初的想法是在 RAG 检索之后加入一个机器学习模块，让模型从 top-k 候选 context 中进一步筛选出更相关的资料。这个思路本身是合理的，因为它不是让机器学习模型直接生成医疗回答，而是让机器学习模型帮助系统“挑材料”。

不过，如果只把我们自己的 ML reranker 与原始 embedding similarity 进行比较，实验说服力不够强。因为 embedding similarity 只是最基础的 baseline。为了让项目更完整，本项目将加入现有开源 reranker 插件作为对比对象。

因此，本项目的最终定位是一个 comparative study：

比较原始 embedding ranking、开源 reranker，以及我们自己训练的 lightweight ML reranker 在医疗问答检索任务中的表现。

我们不会简单地声称自己的方法一定最好，而是从排序效果、运行效率、可解释性和实现成本等角度分析不同方法的优势与不足。

四、研究目标

本项目的主要目标包括：

1. 构建医疗 question-context relevance 数据集
将医疗问题与候选医疗资料配对，并构造 relevant / not relevant 标签。
2. 实现基础 embedding retrieval baseline
使用 embedding similarity 从医疗问答数据库中召回 top-k 候选 context。
3. 接入至少一个开源 reranker 插件
例如 BGE Reranker 或 Sentence-Transformers Cross-Encoder，用于对同一批候选 context 重新排序。
4. 训练一个轻量级 ML reranker
使用 Logistic Regression、Random Forest、SVM 等传统机器学习模型，根据可解释特征预测 question-context relevance。
5. 统一比较不同 reranking 方法
使用 Precision@K、MRR、NDCG@K、运行时间等指标比较不同方法。
6. 分析我们方法的优势与不足
重点分析轻量级 ML reranker 在医疗检索场景中的可解释性、速度和改进空间。

五、研究问题

本项目主要回答以下问题：

Research Question 1:

轻量级机器学习模型能否根据 question-context 特征判断医疗问题与候选资料之间的相关性？

Research Question 2:

与原始 embedding similarity 排序相比，开源 reranker 和 lightweight ML reranker 是否能提升医疗 context 的排序质量？

Research Question 3:

与开源 neural reranker 相比，轻量级 ML reranker 在准确性、速度和可解释性方面分别有什么优势和不足？

Research Question 4:

在医疗问答检索任务中，哪些特征对 relevance prediction 更有帮助？例如 semantic similarity、keyword overlap、department match 等。

六、数据来源与样本构造

本项目使用已有中文医疗问答数据作为实验基础，主要包括儿科和肿瘤科相关问答数据。每条原始数据通常包含患者问题、医生回答、科室或疾病信息等字段。

为了适配机器学习任务，本项目会将原始数据整理成 question-context pair 的形式：

question	candidate_context	label
某个医疗问题	与该问题对应的医生回答	1
某个医疗问题	随机抽取的其他科室回答	0
某个医疗问题	同科室但不同疾病的回答	0

其中，label = 1 表示该 context 与 question 相关，label = 0 表示不相关。

正样本

原始医疗问答中，问题本来对应的医生回答可以视为相关 context，因此原始 question-answer pair 将作为正样本。

普通负样本

从不同科室或明显不同疾病的问题回答中随机抽取 context，与当前问题配对。例如，儿童发烧问题配上肿瘤化疗相关回答，通常可以视为不相关样本。

困难负样本

从同一科室但不同疾病的问题回答中抽取 context，构造 hard negative。例如，同样是儿科，一个问题儿童发烧，另一个 context 是儿童腹泻。这类样本可以避免模型只依赖科室信息进行简单判断。

七、方法设计

本项目将比较三类方法。

方法一：Embedding Similarity Baseline

这是最基础的检索排序方式。系统将 question 和 context 转换成 embedding，然后计算 cosine similarity，并按相似度从高到低排序。

该方法的优点是速度快、实现简单，适合作为基础 baseline。缺点是它对 question 和 context 的细粒度交互建模不足，在医疗场景中可能会受到相似症状、相似表达、科室交叉等因素影响。

方法二：开源 Reranker 插件

本项目计划接入至少一个开源 reranker，例如：

- BGE Reranker
- Sentence-Transformers Cross-Encoder

这类模型通常会把 question 和 candidate context 同时输入模型，并直接输出 relevance score。相比 embedding similarity，它们能够更充分地建模 question-context 之间的语义关系，因此一般具有更强的重排序能力。

由于项目时间较紧，本项目的最低目标是完成一个开源 reranker 的接入与实验。如果时间允许，再额外加入第二个开源 reranker 作为扩展对比。

方法三：Our Lightweight ML Reranker

我们将训练一个轻量级机器学习模型，用于预测 question-context pair 是否相关。

输入特征包括：

1. Semantic Similarity
question 和 context 之间的 embedding cosine similarity。
2. Keyword Overlap
question 和 context 中关键词或重要词汇的重合比例。
3. Department Match
question 和 context 是否来自同一科室或疾病类别。
4. Question Length
question 的文本长度。
5. Context Length
candidate context 的文本长度。
6. Original Retrieval Score
原始 embedding retrieval 阶段得到的相似度分数。

候选模型包括：

- Logistic Regression
- Decision Tree

- Random Forest
- SVM

其中，Random Forest 将作为主要模型之一，因为它训练速度较快，效果相对稳定，并且可以提供 feature importance，便于解释模型判断依据。

八、实验流程

本项目的实验流程如下：

第一步，建立独立 GitHub repository。

新 repo 只保留 CPS3830 项目需要的代码、数据样例、实验脚本、结果表格和报告内容。原 RAG 项目只作为数据来源和背景参考，不作为本课程项目的直接提交主体。

第二步，整理医疗问答数据。

从原始 CSV 数据中提取 question、answer、department 等字段，进行清洗和格式统一。

第三步，构造 question-context pair dataset。

为每个 question 构造正样本、普通负样本和 hard negative samples，生成可用于监督学习的数据集。

第四步，运行 embedding retrieval baseline。

使用 embedding similarity 从候选资料库中召回 top-k context，例如 top-10 或 top-20。

第五步，运行开源 reranker。

使用 BGE Reranker 或 Cross-Encoder 对同一批 top-k candidates 进行重排序，保存每个 candidate 的 relevance score 和排序结果。

第六步，训练 lightweight ML reranker。

提取 semantic similarity、keyword overlap、department match、text length 等特征，训练 Logistic Regression、Random Forest、SVM 等模型。

第七步，统一评估不同方法。

在同一批 question 和同一批 candidate context 上比较 embedding baseline、开源 reranker 和 lightweight ML reranker。

第八步，整理结果与分析。

生成实验表格，比较不同方法的排序质量、运行时间和可解释性，并总结我们的方法哪里有优势、哪里需要改进。

九、评估指标

本项目将从排序效果、分类效果和运行效率三个角度进行评估。

1. 排序效果指标

Precision@K

衡量前 K 个候选 context 中有多少是真正相关的。

Recall@K

衡量真正相关的 context 是否出现在前 K 个结果中。

MRR (Mean Reciprocal Rank)

衡量第一个 relevant context 出现的位置。相关 context 越靠前，MRR 越高。

NDCG@K

衡量整体排序质量。如果后续引入多级相关性标签，可以使用该指标。

考虑到项目时间有限，本项目会优先使用 Precision@K 和 MRR。如果时间允许，再加入 NDCG@K。

2. 分类效果指标

对于 lightweight ML reranker，会额外评估二分类效果：

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

这些指标用于分析模型是否能正确判断 question-context 是否相关。

3. 运行效率指标

由于开源 neural reranker 通常比传统 ML 模型更慢，本项目会记录运行时间：

- Average reranking latency per query
- Average scoring time per question-context pair

这可以帮助我们分析不同方法在实际系统中的部署成本。

十、预期结果

本项目预期得到以下结果：

1. Embedding similarity baseline 速度最快，但排序质量可能有限。
2. 开源 reranker 可能具有最强的排序效果，但运行成本较高，可解释性较弱。
3. Lightweight ML reranker 的效果不一定超过开源 reranker，但它具有训练快、部署轻量、特征可解释、容易加入医疗领域特征等优势。
4. 如果 lightweight ML reranker 的效果接近开源 reranker，同时速度更快，则说明它在小规模医疗问答检索场景中具有一定实用价值。
5. 如果 lightweight ML reranker 明显弱于开源 reranker，也可以通过实验分析原因，例如数据量不足、hard negative 不够、手工特征表达能力有限等，并提出后续改进方向。

本项目的重点不是证明我们的方法一定最强，而是通过公平对比分析不同 reranking 方法在医疗问答检索任务中的表现。

十一、项目可行性

本项目具有较高可行性，原因如下：

第一，数据基础已经具备。现有医疗问答数据可以直接用于构造 question-context pair，不需要从零开始收集数据。

第二，任务边界清晰。本项目聚焦于 retrieval reranking，不需要重新开发完整 RAG 系统。

第三，模型复杂度可控。传统 ML 模型训练速度快，可以在短时间内完成；开源 reranker 主要用于推理和对比，不一定需要从零训练大模型。

第四，学校提供 GPU 资源。开源 reranker 的运行可以使用学校显卡完成，能够降低实验时间压力。

第五，项目可以采用最小可行版本。最低目标是完成三方对比：embedding baseline、一个开源 reranker、our lightweight ML reranker。如果时间充足，再扩展第二个开源 reranker 或更多评估指标。

十二、时间计划

由于项目时间较紧，计划采用最小可行实验方案。

第一阶段：当天晚上

1. 新建独立 GitHub repo
建立清晰项目结构，例如 data、src、experiments、results、report 等目录。
2. 整理数据格式
从医疗问答数据中提取 question、answer、department 字段，生成统一的 CSV 或 JSONL 文件。
3. 构造初版 question-context pair dataset
为每个 question 生成正样本、普通负样本和部分 hard negative。
4. 完成 embedding similarity baseline
生成原始 top-k 检索结果，作为所有 reranker 的共同输入。
5. 测试 GPU 环境
确认学校显卡可以运行开源 reranker，例如 BGE Reranker 或 Cross-Encoder。

第二阶段：周一上午

1. 接入开源 reranker
优先完成一个开源 reranker 的运行，例如 BGE Reranker 或 Cross-Encoder。
2. 运行开源 reranker 实验
对同一批 top-k candidates 进行 reranking，保存 relevance scores 和排序结果。
3. 提取 lightweight ML features
计算 semantic similarity、keyword overlap、department match、question length、context length 等特征。

第三阶段：周一下午

1. 训练 lightweight ML reranker
训练 Logistic Regression、Random Forest、SVM 等模型。
2. 评估 classification performance
计算 Accuracy、Precision、Recall、F1-score 和 Confusion Matrix。
3. 评估 reranking performance
计算 Precision@K、MRR，并与 embedding baseline、开源 reranker 进行对比。

第四阶段：周一晚上

1. 整理实验结果
生成结果表格和简单图表。
2. 分析不同方法优缺点
比较不同方法在准确性、速度和可解释性方面的差异。
3. 完成报告与展示内容
准备最终 submission 所需的 report、slides 和 demo 说明。

十三、项目风险与应对

风险一：开源 reranker 环境安装失败

应对方案：

优先选择文档清晰、社区使用较多的模型。如果 BGE Reranker 运行失败，可以切换到 Sentence-Transformers Cross-Encoder；如果两个都遇到问题，则至少保留一个可运行的开源 reranker 作为对比目标。

风险二：时间不足，无法比较多个开源插件

应对方案：

最低完成版本只需要比较三类方法：embedding baseline、一个开源 reranker、our lightweight ML reranker。第二个开源 reranker 作为 optional extension。

风险三：我们的 ML reranker 效果不如开源 reranker

应对方案：

这不代表项目失败。报告中可以分析原因：开源 reranker 经过大规模预训练，而我们的模型只使用少量手工特征。我们的优势可以放在轻量、可解释、速度快和易部署上。

风险四：自动构造标签不够准确

应对方案：

在报告中说明本项目使用原始问答对应关系和负样本构造形成弱监督标签。未来可以加入人工标注或医学专家评估作为改进方向。

风险五：GPU 使用时间有限

应对方案：

控制实验规模，例如先使用部分数据和 top-10 candidates。轻量 ML 模型可以在 CPU 上训练，GPU 主要用于开源 reranker 推理。

十四、预期贡献

本项目的预期贡献包括：

1. 将医疗问答检索问题转化为一个可训练、可评估的机器学习 reranking 任务。
2. 构建一个包含正样本、普通负样本和 hard negative 的 question-context relevance dataset。
3. 实现并比较 embedding similarity baseline、开源 reranker 和 lightweight ML reranker。
4. 从准确性、效率和可解释性三个角度分析不同 reranking 方法的优缺点。
5. 为未来医疗 RAG 系统中的 context selection 模块提供实验基础。

十五、总结

本项目研究的是医疗问答检索阶段的重排序问题。相比直接构建完整 RAG 系统，本项目更加聚焦于 CPS3830 Machine Learning 的核心内容：数据构造、特征工程、模型训练、模型评估和结果分析。

项目将新建独立 GitHub repository，与原有 RAG 项目保持边界清晰。实验将比较原始 embedding similarity、开源 reranker 和我们自己训练的 lightweight ML reranker。通过统一实验设置，本项目希望分析不同 reranking 方法在医疗问答检索任务中的表现，并进一步理解轻量级机器学习方法在实际检索增强系统中的价值与局限。

考虑到项目时间有限，本项目会采用最小可行实验策略，优先保证完整流程跑通，再根据时间扩展更多模型和指标。